

이승준 Backend Portfolio

대용량 비동기 파이프라인을 “돌아가는 코드”에서 “복구 가능한 운영 시스템”으로

Kotlin · Spring Boot · Kafka · MinIO · PostgreSQL · Airflow 2026-08-01 · GitHub [zbnerd](#)

저는 처리량 숫자 하나보다 그 숫자가 나온 조건, 장애가 발생하는 경계, 다시 실행했을 때의 동작을 함께 설명합니다.

이 문서에서 보여주는 5가지 문제 해결

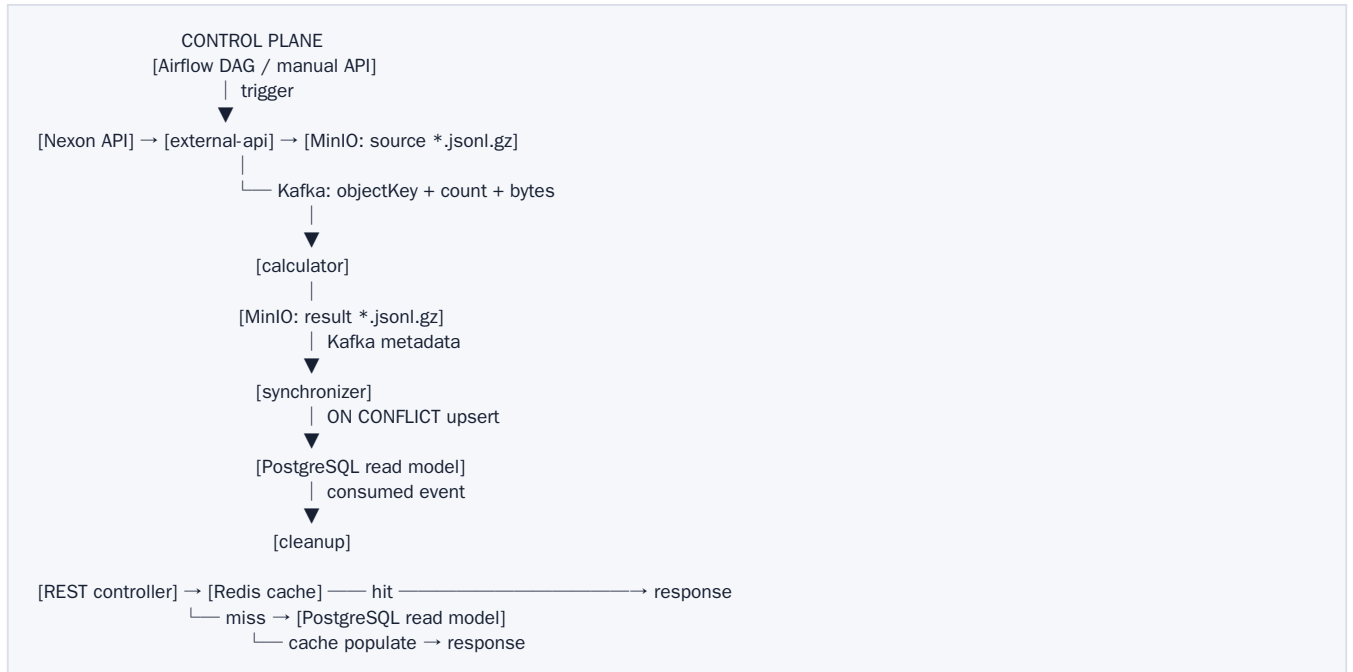
1. 큰 payload 결합을 Kafka claim-check ETL로 분리
2. at-least-once 전달을 artifact-ACK-upsert 먹등 경계로 흡수
3. pipe streaming data-loss race를 temp-file async upload로 교체
4. 보고서상 약 71시간 관측에서 드러난 dual orchestration race를 코드까지 추적
5. 보고서상 약 80시간 관측 창으로 single-writer 병목 경계를 식별하고 후속 검증 과제 정의

Project Overview

게임 캐릭터 장비 가치 산정 파이프라인

개인 프로젝트 · Backend 1명 · 2026.05~진행 중

외부 API에서 수집한 수십~수백 KB 장비 응답을 확률 계산을 하고, 조회용 read model로 만드는 비동기 ETL입니다. Airflow는 실행 순서를 제어하고, Kafka는 작은 이벤트를 전달하며, 실제 대용량 데이터는 MinIO의 gzip JSONL artifact로 이동합니다.



위 도식은 standalone cleanup까지 분리된 현재 4단계 구조입니다. 아래 82시간 검증 시점은 그 이전의 3서비스 구조였습니다.

제 역할

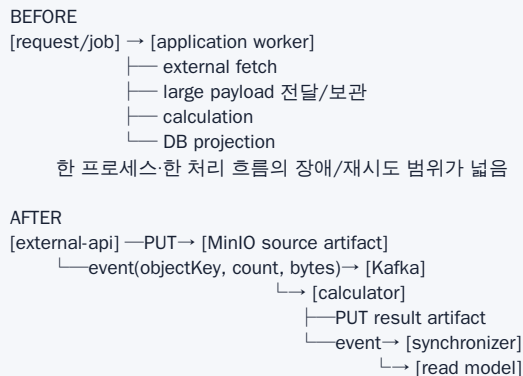
- 문제 정의, 데이터/실패 경계 설계, 구현·PR acceptance, 부하/장기 테스트 기준과 결과 판정을 담당했습니다.
- AI는 탐색·초안·반대 관점 수집에 사용했습니다. AI 기록의 “완료”나 숫자는 증거로 승인하지 않고 실제 diff-테스트·로그·지표로 다시 확인했습니다.

증거 읽는 법

- Observed: 날짜·환경·부하 조건과 함께 실제 보고된 결과
- Target/Estimate: 목표 또는 추정. 성과 숫자로 사용하지 않음
- Latest merged ref: 로컬 HEAD보다 뒤지만 GitHub merge와 로컬 reachable ref가 모두 확인된 변경은 ref를 명시

Case 1. 큰 payload 결합을 Kafka claim-check ETL로 분리

Before → After



문제와 원인

- 외부 API 원문, 계산, DB 투영이 한 실행 흐름에 붙어 있으면 큰 payload가 queue/broker와 JVM 메모리에 압력을 주고, 한 단계 실패가 전체 재실행으로 번집니다.
- 브로커에 본문을 넣는 선택은 message size·재전송 비용을 키우고, DB에 raw staging을 먼저 쓰는 선택은 connection pool과 write amplification을 수집 속도에 결합합니다.

결정과 구현

- Claim-check: gzip JSONL 청크를 object storage에 저장하고 Kafka에는 **objectKey**, record count, compressed/uncompressed bytes 등 작은 metadata만 전송했습니다.
- 82시간 검증 시점에는 external-api, calculator, synchronizer를 별도 프로세스로 나눴고, **CHUNK_CONSUMED** cleanup은 external-api 내부 **ConsumedChunkCleanupScheduler**가 담당했습니다.
- standalone **module-cleanup**은 2026-06-07 이후 분리됐습니다. 현재 4단계 구조는 이 후속 진화를 포함하므로 82시간 결과와 시점을 합치지 않았습니다.
- **ArrayBlockingQueue**와 single writer로 producer fan-out을 유한한 메모리 경계에 모으고, 청크 단위 manifest/marker로 완료 상태를 표현했습니다.

검증 결과

2026-05-23 22:53~05-27 09:08, external-api/calculator/synchronizer 3모듈, 각 -Xms512m -Xmx1g 조건의 82h 15m 실행에서:

users	items	chunks	input(uncompressed)	restart / ERROR logs
60,190,417	4,034,907,241	120,442(실패 0)	13.31 TB	0 / 0

48~82시간 total RSS는 3,546~3,687 MB로 34시간 drift가 4% 미만이었습니다.

한계

이 결과는 당시 3서비스·host process 배포와 external-api 내부 cleanup의 보고서 기반 관측입니다. 이후 standalone cleanup/Docker 4서비스 환경의 성능과 동일시하지 않으며, Kafka broker HA를 검증했다는 주장도 하지 않습니다. [E-002, E-005]

Case 2. at-least-once 전달을 단계별 먹등 경계로 흡수

실패해도 다시 시작할 수 있는 순서



문제와 원인

- Kafka consumer는 처리 중 crash, ACK 실패, 재balance로 같은 event를 다시 받을 수 있습니다.
- object upload보다 이벤트가 먼저 보이면 consumer가 존재하지 않는 key를 읽고, 결과 파일은 있는데 event만 유실되면 재계산 또는 영구 누락이 발생합니다.
- DB에 단순 INSERT를 하면 재전달이 중복 row 또는 constraint error로 바뀝니다.

결정과 구현

- source/result artifact key를 run/endpoint/chunk로 결정해 동일 event의 결과 위치가 바뀌지 않게 했습니다.
- result가 이미 있으면 계산을 반복하지 않고 result-ready event만 다시 발행합니다.
- upload future가 성공한 callback에서만 chunk-ready를 발행하고, 실패하면 publish를 생략해 “event가 보이면 object도 보인다”는 순서를 지켰습니다.
- calculator dispatch 성공 뒤 manual ACK, synchronizer에는 ON CONFLICT (read_key) DO UPDATE와 document hash 비교를 적용했습니다.

검증과 결과

- 82시간 실행의 calculator chunk 실패 0과 120,442 processed chunks는 당시 claim-check core의 프로젝트 관측치입니다. 현재 upload-success callback 등은 2026-06월 후속 변경이므로 이 수치를 현 경계의 장기 회귀 결과로 사용하지 않습니다.
- 현재 경계는 coordinator/dispatch/upsert focused test와 실제 merged diff로 duplicate/retry 순서를 확인했습니다. 이 조합 이후 동일한 82시간 장기 run은 별도 근거로 남아 있지 않습니다.

한계

Exactly-once가 아닙니다. at-least-once 전달에서 재실행을 안전하게 만드는 설계입니다. 또한 로컬 HEAD의 cleanup inbox는 in-memory/drop-oldest여서 durable end-to-end라고 표현하지 않습니다. durable cleanup inbox는 이후 merged PR #1463 범위로 따로 구분합니다. [E-003, E-004]

Case 3. pipe streaming data-loss race를 correctness-first로 교체

더 “우아한” streaming이 0-byte gzip을 만든 과정

FAILED DESIGN (2026-06-20)

```
producer coroutine → GZIPOutputStream → PipedOutputStream
                    <-> lifecycle race
AWS SDK background reader ← PipedInputStream
    ↳ "Read end dead" / closed Deflater
    ↳ 0-byte or truncated result artifact
```

CORRECTIVE DESIGN (2026-06-22)

```
Flow.collect → gzip temp file → flush + close
              ↓
              putFileAsync(S3 TransferManager)
                | completion
                ↓
              deleteIfExists(temp)
```

문제와 원인

- item-equipment 결과가 0-row **result-part-*.jsonl.gz**로 저장돼 synchronizer가 valuation row를 받지 못하고 DAG 완료 조건도 충족하지 못했습니다.
- producer coroutine이 끝난 시점과 SDK가 pipe를 읽는 background thread의 시점이 달랐습니다. **PipedInputStream**은 writer thread lifecycle을 보고 **Read end dead**를 던졌고, completion callback은 drain 도중 pipe를 닫을 수 있었습니다.
- 메모리와 disk I/O를 줄이려던 streaming 최적화가 artifact correctness를 깨뜨린 사례였습니다.

결정과 trade-off

- pure pipe streaming을 폐기하고 bounded gzip temp file을 완전히 닫은 뒤 비동기 file upload를 시작했습니다.
- extra disk write/read를 감수해 producer/consumer lifecycle을 분리하고, upload 성공·실패 어느 쪽이든 temp file을 정리했습니다.
- chunk는 최대 500 records/128 MiB uncompressed로 제한해 disk pressure의 상한을 두었습니다.

검증 결과 — 2026-06-22 MinIO E2E

내부 ITEM_EQUIPMENT phase를 **runId=verify-writer-fix-1**로 실행하고 Prometheus counter, calculator log, MinIO object download, **gunzip -c**/line count로 확인했습니다.

관측	Before	After
processed chunks	0	64
failed chunks	-	0
calculated items	-	1,948,957
Read end dead	약 14,921 / 9분	0
result artifact	0 bytes, invalid	674,986 bytes, 30,507 valid rows

한계

수치는 Accepted ADR-730에 보존됐고 raw log bundle 전체는 checkout에 없습니다. 따라서 “속도 향상”이 아니라 실패 설계를 철회하고 데이터 정합성을 회복한 결과로 사용합니다. failed commit **85b5528d**와 corrective merged PR #1325/commit **205ce14b**를 함께 남겼습니다. [E-004A]

Case 4. 보고서상 약 71시간 관측에서 드러난 dual orchestration race 추적

두 제어면이 같은 slot을 잡은 사건

03:00 KST BEFORE

```
[legacy @Scheduled] → ITEM_EQUIPMENT once → same phase slot
[Airflow morning_chain] → infinite loop → "slot occupied" → loop STOPPED
```

AFTER

```
[Airflow morning_chain] → single phase owner
[manual endpoint] → operator-triggered recovery only
[legacy @Scheduled] → X removed
```

문제

원본 보고서는 이 실행을 ~71h로 표기하지만, 기재된 2026-06-23 09:03~06-26 05:30은 68h27m입니다. 이 관측 창에서 보고서는 인프라 restart/OOM/Kafka-DB 장애 0, 2.28B items와 calculator error 0을 기록했습니다. 그러나 06-25와 06-26 03:00에 처리 loop가 중단돼 수동 restart가 필요했고, 150시간 무개입 목표는 달성하지 못했습니다. 두 중단 중 slot race가 로그·타임라인으로 직접 확인된 것은 06-26입니다.

원인 추적

- 06-25의 최초 “null upstream” 설명은 당시 직접 확정하지 못했습니다. 06-26의 로그와 code path로 dual orchestration 원인을 확인한 뒤, 06-25 중단도 같은 원인의 다른 발현으로 소급 추정했습니다.
- 06-26에는 legacy Spring @Scheduled 4-phase chain이 ITEM_EQUIPMENT once-run slot을 먼저 점유했고, 34초 뒤 Airflow morning_chain의 infinite loop가 같은 slot에서 slot occupied로 종료되는 타임라인을 직접 확인했습니다.
- once-run이 1,734 chunks를 끝내도 infinite loop를 다시 시작할 주체가 없어 pipeline이 IDLE로 남았습니다.

결정과 trade-off

- 실행 주체를 Airflow로 단일화하고 legacy cron을 제거했습니다(ADR-736, merged PR #1433).
- 중복 오케스트레이터를 “fallback”으로 두지 않았습니다. slot race를 없애는 대신 Airflow가 자동 실행의 단일 장애점이 되는 비용을 명시적으로 수용했습니다. 수동 trigger endpoint는 복구 수단으로 유지했습니다.

결과와 한계

- compile과 기존 scheduler test는 통과했고 dual-fire source는 코드에서 제거했습니다.
- 다음 03:00의 연속 runtime 성공은 ADR에 deferred로 기록돼 있습니다. 따라서 “재발 0”이 아니라 “원인 제거 구현, 운영 재검증 필요”가 정확한 결론입니다. [E-006, E-007]

Case 5. 보고서상 약 80시간 관측 창으로 single writer 병목 식별

리소스 포화가 아니라 queue drain이었다



문제와 관측 조건

- 2026-06-29~07-02 ITEM_EQUIPMENT phase, 보고서상 약 80시간 관측 창, Contabo 8 cores/23GB RAM/600Mbps
- external-api-calculator uptime 80h14m, synchronizer-cleanup 71h11m; 4 containers + PostgreSQL/MinIO/Kafka/Airflow
- ITEM_EQUIPMENT max connections/rate/in-flight 250, snapshot queue capacity 3,000

가설 비교와 결정

후보	관측	판정
Nexon/API failure	38.08M fetched 중 294 failures(0.0008%)	주 병목 아님
Memory/GC	23GB 중 7GB free, Full GC 0	주 병목 아님
CPU/network	여유가 남은 시점에도 queue 포화	단독 설명 불가
Writer drain	queue 3,000, submit 1.3~1.7s, 35초 뒤 ~525	병목 근거

결론과 변경 시점

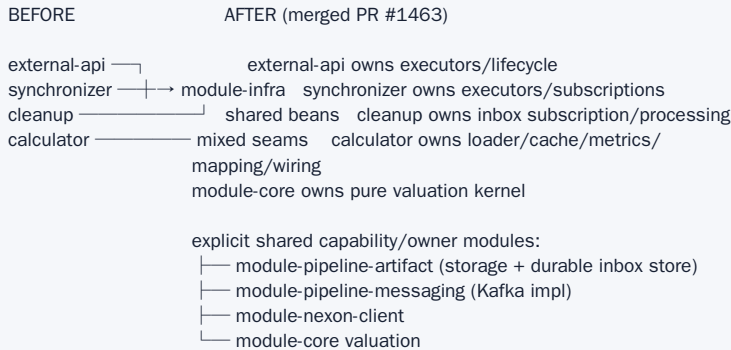
- observed sustained는 100~150 users/s, report-listed burst max는 651.7이었습니다. 보고서의 lifetime average는 38.08M/80h14m 산술과 맞지 않아 제외했습니다.
- 35초 동안 queue depth가 약 2,475 줄어든 값은 처리량이 아니라 queue-depth 순감소율 약 71 records/s로 표기했습니다.
- commit ancestry를 대조하면 producer-side serialization(**b733a8dfb**)과 upload-success callback(**ecee74549**)은 이 관측 전에 이미 적용돼 있었습니다. 따라서 이 실행은 두 변경의 before benchmark가 아닙니다.
- 해당 구성에서도 한 consumer가 맡은 gzip append/chunk close:async upload 등록 경계에서 queue가 포화됐습니다. 보고서 뒤에는 gzip 기본 level만 **BEST_SPEED(7c120d647)**로 바뀌었습니다.

한계와 후속 검증

이 실행은 진단(no code changes applied)입니다. ADR-729의 ≥ 150 files/s는 target이고 사후 측정값은 기록되지 않았습니다. PR #1452의 Kafka/MinIO tuning은 closed-unmerged이며, **BEST_SPEED** 뒤 동일 조건 after도 없습니다. 따라서 성과는 “병목 경계 식별”과 “후속 변경의 검증 과제 명시”까지입니다. Airflow scheduler 359 restarts(password drift)도 전체 무장애 표현에서 제외했습니다. [E-008, E-009]

Latest Architecture Follow-up — worker-owned runtime

소유권이 흐린 실행 자원



문제와 원인

- worker가 공용 `module-infra`의 executor, lifecycle, storage bean을 우연히 가져오면 실제 runtime owner가 코드 경계와 달라집니다.
- bean 후보 수-classpath 변화가 startup을 깨뜨리고, shutdown/metric/retry 정책을 어느 모듈이 책임지는지 불명확해집니다.

결정과 구현

- artifact identity/storage/lifecycle, Kafka delivery outcome/retry/DLT, valuation kernel, Nexon client를 명시적 모듈로 추출했습니다.
- external-api와 synchronizer가 자기 executor-subscription-lifecycle을 직접 소유하게 했습니다.
- `verifyActiveEtlInfrasolation` build gate와 source/runtimeClasspath guards를 추가해 worker가 다시 `module-infra`를 직접 참조하면 build에서 실패하게 했습니다.

검증 결과

2026-07-20 merged PR #1463의 final tip과 tip 직전 closure evidence를 분리해 확인했습니다.

항목	결과
active worker의 direct <code>module-infra</code> runtime dependency	final tip <code>1f47173e3</code> 에서 3 → 0
named focused tests	tip 직전 <code>11ee3c727</code> : 86 passed / 0 failed / 0 skipped
executable packaging	tip 직전: 4 worker bootJar 성공
dependency/source guard	tip 직전: 위반 0, gate 통과

한계

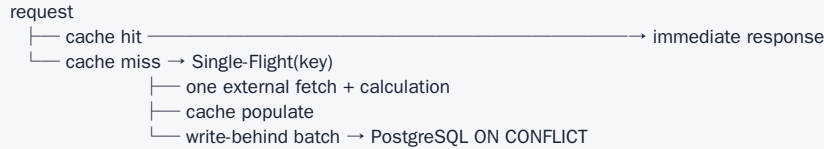
이 ref는 로컬 HEAD 이후 42개 commit이며 GitHub상 develop에 merge된 PR입니다. 다만 closure evidence 뒤 final tip에서 production subscription class 4개와 test file 2개가 다시 변경됐고, 이 마지막 수정을 포함한 test/bootJar 재실행 증거는 없습니다. full repository/root check, Testcontainers, load/performance test도 생략했습니다. 따라서 final tip의 구조 상태와 tip 직전 focused regression을 분리한 성과이며, 처리량 개선 성과가 아닙니다. [\[E-010\]](#)

Earlier Project & Open Source

조회 API의 cache fast path와 write path 분리

2025.11~2026.04 · 개인 프로젝트

고비용 외부 API/확률 계산을 모든 조회가 반복하던 구조에서 cache hit와 miss를 분리하고, 동일 key Single-Flight와 write-behind batch upsert를 적용했습니다. 계층형 캐시 PR #83, Single-Flight #263, write-behind #266, production batch upsert #618의 실제 merged diff를 각 기능과 연결했습니다. [E-011A]



2026-03-24 프로젝트 문서는 DB 200k~300k rows, `wrk -t4 -c200 -d120s` 후속 측정으로 7,347 RPS, p99 36ms, errors 0을 기록합니다. 하지만 원시 wrk stdout과 전체 환경 파일이 없고 최초 측정에는 errors 65 기록이 있어 제한부 증거입니다. 조건이 다른 97 RPS와 합쳐 76배라고 표현하지 않았습니다. [E-011]

Resilience4j Kotlin TimeLimiter

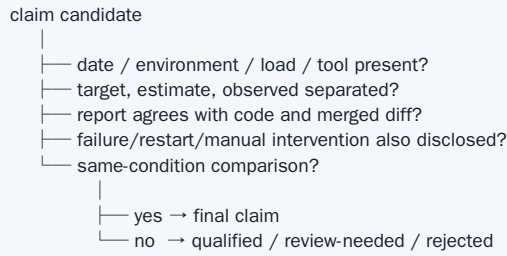
공개 issue #2307에서 일반 함수용 Kotlin TimeLimiter API가 빠진 것을 확인하고 `decorateFunction·executeFunction`과 성공/실패/timeout 5개 테스트를 구현했습니다.

upstream 결과	값
PR	resilience4j #2407
merge	2026-03-06
diff	2 files, +163/-0(main +28, test +135)
정확한 기여 표현	기존 공개 이슈를 구현-테스트로 해결

최초 문제 제기자는 다른 contributor이므로 “직접 최초 발견”으로 쓰지 않습니다. [E-012]

Verification Strategy

숫자를 채택하는 기준



조사 범위

Source	전수 결과	사용 원칙
Local Git, all refs	2,342 unique commits	root/normal/merge를 empty-tree/parent/first-parent diff로 통일
GitHub PR	709/709	번호·상태·merge metadata 전수; 500개 상세, 209개 상세 inaccessible; 핵심 diff 별도 검증
GitHub issue	752/752	merged closing PR 없으면 해결 미검증
Original PDFs	31+2+1 pages	34쪽 구조·텍스트·render 전수 검사, 원본 보존
AI traces	실제 docs/ai-traces 재귀 조사	세션 문구는 비신뢰, 코드·Git으로 재검증

GitHub의 마지막 209개 PR 상세 GraphQL 호출은 반복 HTTP 502로 review/discussion/commit/link/file metadata를 얻지 못했습니다. 번호·상태·merge metadata는 REST로 보존했고, 본문 핵심 PR은 로컬 diff 또는 개별 API로 재확인했습니다.

대표적으로 버린 문구

- 97 → 7,347 RPS, 76배: 동일 조건 아님
- 150h 무인 안정성: 보고서상 71h(기재 시각 68h27m) 종료와 수동 개입 기록
- ADR-729 목표 ≥ 150 files/s 달성: 사후 측정 기록 없음
- small-file 10~100배 감소: ADR-743 proposed, 구현 범위 밖
- exactly-once: 실제 설계는 at-least-once + idempotent boundaries

How I Work

운영 문제를 다루는 순서

1. 재현 조건 고정: 시간, ref, 서버 자원, 데이터량, concurrency, 도구를 먼저 기록합니다.
2. 경계 단위 가설: HTTP → queue → storage → Kafka → compute → DB 중 어디에서 대기·중복·유실이 생기는지 나눕니다.
3. 반증 지표 선택: throughput 하나가 아니라 queue depth, submit latency, lag, ACK, error counter, RSS/GC, DB wait를 함께 봅니다.
4. trade-off 기록: Airflow SPOF, bounded queue의 거부, artifact 재처리 비용처럼 해결책이 새로 만드는 위험을 ADR에 남깁니다.
5. 완료 조건 분리: compile/test, runtime smoke, 장기 관측, 동일 조건 성능 재측정을 서로 다른 단계로 관리합니다.

AI 활용 원칙

- AI에게 코드 위치 탐색, 가설 후보, 반대 검토, 테스트 아이디어를 요청합니다.
- destructive DB 작업·비밀 출력·근거 없는 성과 문구는 acceptance 대상에서 제외합니다.
- 최종 판단은 actual diff, 실행 결과, 로그/metric, source state를 기준으로 합니다.

면접에서 먼저 설명할 한계

- 82h/보고서상 71h/보고서상 80h 관측은 날짜·배포·모듈·환경이 다르며 직접 합산·비교하지 않습니다.
- 71h로 표기된 보고서의 기재 시각은 68h27m이고, orchestration 수정은 코드/test 검증까지이며 다음 03:00 연속 runtime 증거가 남았습니다.
- 약 80h 관측 창은 서비스별 uptime이 다르며, single-consumer writer 경계를 식별했지만 후속 **BEST_SPEED**의 동일 조건 after benchmark가 없습니다.
- PR #1463의 86-test/4-bootJar 근거는 tip 직전이며, final-fix-inclusive rerun-full check·load test는 없습니다.
- 과거 7,347 RPS는 프로젝트 문서 기반이고 raw output 보존이 부족합니다.

Links

- Project: github.com/zbnerd/probabilistic-valuation-engine
- Open source: [Resilience4j PR #2407](#)
- Detailed claim map: [output/research/evidence_ledger.md](#)

좋은 운영 결과는 “문제가 없었다”만이 아니라, 문제가 생겼을 때 어디까지 안전했고 무엇을 아직 검증하지 못했는지 설명할 수 있는 상태라고 생각합니다.